Emergent Axial Lisp

A Homoiconic Multi-Axial Declarative Language for the Meta-CONS Runtime

Formal Architecture, Type System, Reflective Metaobject Protocol, Quasigroup Recovery Model, and Pure Haskell Reference Design

Status: Foundational White Paper / Normative Architecture Draft
Language: Emergent Axial Lisp
Runtime: Meta-CONS Runtime
Type Architecture: Multi-Axial Runtime Model
Reflective Protocol: Meta-CONS Metaobject Protocol
Foundational Protocol: OMI / Omicron / Omnicron
Reference Implementation Language: Haskell
Portable Execution Targets: Omnicron ISA, portable C, WebAssembly, embedded runtimes, and other bounded carriers

---

Abstract

This paper introduces **Emergent Axial Lisp**, a homoiconic multi-axial declarative language evaluated by the **Meta-CONS Runtime**.

Emergent Axial Lisp is designed for declarations whose meaning cannot be determined by lexical syntax alone. A form is resolved through several independent but cooperating axes:

```
scope
structure
binding
origin
evaluation policy
compilation stage
temporal position
integrity
carrier position
projection
capability
```

The language preserves canonical program structure as S-expressions while permitting a readable M-expression surface that lowers deterministically into canonical CONS forms. Unlike a conventional Lisp environment in which a symbol is mapped directly to one value or one function, the Meta-CONS environment resolves a symbol through typed coordinates including:

```
FS / GS / RS / US
CAR / CDR / CONS
APVAL / APVAL1
SUBR / FSUBR
EXPR / FEXPR
LL / MM / NN
LOGOS / NOMOS / PATHOS
origin-tagged module contribution
affine / projective carrier position
```

Independent local or remote `.o` knowledge sources contribute sparse, origin-tagged 256-bit boards. These contributions enter an origin-preserving coproduct rather than a single mutable namespace. Meta-CONS composes compatible contributions into a shared 256-position OMI-Lisp plane while preserving the full ordered CAR/CDR relation and provenance behind each visible coordinate.

The runtime further defines a quasigroup recovery law. Given any two coordinates among CAR, CDR, and CONS, the missing coordinate can be recovered when the selected resolver profile satisfies the quasigroup laws. This transforms CONS from a mere pair constructor into a bounded relational recovery protocol.

The reference architecture is specified using pure typed Haskell. Closed coordinate families are represented by algebraic data types, promoted kinds, GADTs, and type families. Type classes are reserved for genuinely open behavior such as resolution, reflection, lowering, projection, and host embedding. A monad transformer stack supplies controlled effects, but it does not define semantic meaning; parsing, elaboration, normalization, resolution, recovery, and lowering remain pure functions.

Emergent Axial Lisp therefore unifies:

```
homoiconic programming
multi-stage metaprogramming
metadata and meta-knowledge
reflective metaobjects
metalogic
meta-circular evaluation
meta-memory
typed coproduct composition
quasigroup recovery
deterministic carrier lowering
```

without collapsing declaration, validation, execution, storage, projection, or proof into a single authority surface.

---

1. Introduction

1.1 The architectural problem

Most programming languages determine meaning primarily through:

```
lexical scope
a symbol table
a type environment
an evaluator
a runtime heap
```

That model works when a program is assumed to exist inside one coherent process with one evaluator and one authoritative environment.

It becomes less adequate when declarations may originate from:

```
independent files

remote peers

embedded devices

formal proof modules

typed rule repositories

human-readable Markdown

graphical canvases

network frames

hardware carriers

historical versions

multiple projection surfaces
```

In such a system, two declarations may use the same visible symbol without having the same:

```
origin
```

```
scope

authority

evaluation policy

temporal position

integrity status

carrier position

projection

capability
```

A conventional environment can preserve some of these properties through records or metadata, but they remain secondary data attached to an otherwise privileged symbol lookup.

Emergent Axial Lisp reverses that relationship.

A declaration is not first a symbol and later a record of context.

A declaration is first a **typed multi-axial coordinate**. Its visible symbol is one projection of that coordinate.

1.2 The central claim

The central claim of this paper is:

> A homoiconic language can remain structurally Lisp while replacing single-axis symbol lookup with typed multi-axial resolution.

The canonical runtime object is not a fixed tuple and not a single host-language object.

It is an emergent relation:

$$R = (\sigma, \kappa, \beta, \omega, \epsilon, \tau, \iota, \chi, \pi, \gamma)$$

where:

$\sigma = \text{scope coordinate}$ $\kappa \quad = \text{structural coordinate}$ $\beta = \text{binding class}$ $\omega \quad = \text{origin coordinate}$ $\epsilon = \text{evaluation policy}$ τ

The runtime meaning of a form emerges from the lawful intersection of these axes.

---

2. System identity

The project is divided into five named responsibilities.

```
LANGUAGE

Emergent Axial Lisp
```

```
SUBTITLE

A Homoiconic Multi-Axial Declarative Language
for the Meta-CONS Runtime
```

```
TYPE ARCHITECTURE

Multi-Axial Runtime Model
```

```
RUNTIME

Meta-CONS Runtime
```

```
REFLECTIVE PROTOCOL

Meta-CONS Metaobject Protocol
```

These names are related, but they are not synonyms.

2.1 Emergent Axial Lisp

Emergent Axial Lisp is the language presented to authors, modules, adapters, and reflective tools.

It owns:

```
canonical S-expression syntax

readable M-expression syntax

quotation and quasiquotation
```

```
macro forms

FEXPR and special-form declaration

typed module forms

metaobject declarations

multi-stage lowering forms

capability request forms
```

It does not own:

```
proof authority

hardware authority

network authority

validation authority

projection authority
```

2.2 Multi-Axial Runtime Model

The Multi-Axial Runtime Model defines the coordinate system through which language objects are interpreted.

It owns:

```
closed axis definitions

legal combinations

type-level state transitions

dispatch selection

recovery profiles

stage transitions
```

```
authority boundaries
```

2.3 Meta-CONS Runtime

The Meta-CONS Runtime executes, resolves, reflects upon, and lowers canonical language objects.

It owns:

```
environment construction

ordered CONS resolution

coproduct composition

quasigroup recovery

macro expansion

staged evaluation

capability mediation

module loading

reflection

host embedding
```

2.4 Meta-CONS Metaobject Protocol

The Meta-CONS Metaobject Protocol exposes the structure by which runtime objects receive meaning.

It allows inspection of:

```
class

scope

binding
```

```
origin

stage

evaluation policy

temporal position

integrity profile

carrier position

recovery law

lowering rule

projection rule

capability requirements
```

Reflection does not automatically grant mutation authority.

---

3. Relation to OMI, Omicron, and Omnicron

3.1 OMI

OMI remains the foundational relational and notation-citation protocol.

OMI defines the doctrine that:

```
declaration is not validation

validation is not projection

projection is not identity

carrier is not authority
```

The historical and current architecture increasingly separates canonical grammar, relational access, typed construction, resolution, and carrier projection rather than treating one language or VM as the whole protocol.

3.2 Omicron

Omicron supplies the bounded character and gauge surface.

The low/high byte geometry is defined by the reversible mirror:

$$mirror(x) = x \oplus 0x80$$

giving:

```
0x00..0x1F
low control coordinate field

0x20..0x7F
low affine readable field

0x80..0x9F
high projective-control mirror field

0xA0..0xFF
high projective readable field
```

The words “affine” and “projective” refer here to carrier-coordinate roles. They do not by themselves assert that these ranges constitute formal finite affine or projective planes without a separately defined incidence law.

3.3 Omnicron

Omnicron supplies the multi-plane resolver architecture.

It coordinates:

```
scope

temporal position

integrity

activation

carrier transformation

projection
```

```
runtime orchestration
```

The execution dependency order must be distinguished from selector or slide-ruler order. An existing exhaustive hexadecimal/XOR audit explicitly separates ruler addresses from execution authority and places validation before projection.

3.4 Emergent Axial Lisp as the language boundary

Emergent Axial Lisp is not another name for OMI or Omnicron.

The dependency is:

```
OMI
relational and notation doctrine

↓

Omicron
bounded gauge and character reachability

↓

Omnicron
multi-plane resolution architecture

↓

Emergent Axial Lisp
homoiconic declarative language

↓

Meta-CONS
reflective typed runtime
```

---

4. Why the language is “Emergent”

Meaning is emergent because no single field contains the complete runtime identity of a declaration.

Consider the symbol:

```
resolve
```

Its meaning may differ by:

```
FS scope versus US scope

system SUBR versus user EXPR

ordinary evaluation versus FEXPR evaluation

local origin versus remote module origin

previous LL state versus present MM state

low affine carrier versus high projective carrier

surface stage versus normalized stage

pure capability versus network capability
```

The runtime does not globally ask:

```
What does the symbol resolve mean?
```

It asks:

```
Which declaration of resolve
is admissible at this coordinate?
```

The result is computed from the active object graph and environment. It is not stored as a universal slot in a fixed historical tuple.

---

5. Why the language is “Axial”

A dimension usually denotes one coordinate in a common numerical space.

The architecture defined here has several coordinate families with different laws and different authority.

They are better described as axes.

5.1 Scope axis

$$Scope = \{FS, GS, RS, US\}$$

```
FS
frame or file scope

GS
group or graph scope

RS
record or relation scope

US
unit or symbol scope
```

These four coordinates carry scope.

They are not parity bits and not runtime stages.

5.2 Structural axis

$$Structure = \{CAR, CDR, CONS\}$$

```
CAR
carried coordinate or selected source component

CDR
continuation, destination, or resolver component

CONS
resolved ordered relation
```

The canonical relation is:

$$CONS(CAR, CDR)$$

This relation is ordered:

$$CONS(a, b) \ / \ CONS(b, a) =$$

unless a specific resolver profile proves otherwise.

5.3 Binding axis

$$Binding = \{APVAL, APVAL1, SUBR, FSUBR, EXPR, FEXPR\}$$

Expanded with structural classes:

$$Binding^{+} = \{CAR, CDR, APVAL, APVAL1, SUBR, FSUBR, EXPR, FEXPR\}$$

The classes mean:

```
APVAL
primary value binding

APVAL1
alternate, auxiliary, or scoped value binding

SUBR
system-defined ordinary function

FSUBR
system-defined special form

EXPR
user-defined ordinary function

FEXPR
user-defined special form
```

Ordinary operators receive evaluated arguments.

Special forms and FEXPRs receive unevaluated forms according to their declared evaluation policy.

5.4 Origin axis

$$Origin = \{Local, Module, Remote, Generated, Recovered, Embedded\}$$

An origin must remain attached to the contribution through composition.

Two values with equal visible payloads but different origins are not necessarily the same runtime object.

5.5 Evaluation axis

$$Evaluation = \{Eager, Lazy, Special, Macro, Reflective, Quoted\}$$

Evaluation policy is distinct from binding authority.

For example:

```
SUBR + Eager

FSUBR + Special

EXPR + Eager

FEXPR + Special

MacroBinding + Macro

DataBinding + Quoted
```

5.6 Stage axis

$$Stage = \{Surface, Parsed, Expanded, Typed, Normalized, Resolved, Lowered\}$$

A form must not silently move backward from a later stage into an earlier stage.

The intended progression is:

```
surface

↓

parsed

↓

expanded

↓

typed

↓

normalized

↓
```

```
resolved

↓

lowered
```

5.7 Temporal axis

$$Delta3 = \{LL, MM, NN\}$$

```
LL
previous temporal position

MM
present temporal position

NN
forward temporal position
```

The corrected doctrine preserves LL/MM/NN as temporal coordinates rather than treating them as deprecated aliases.

5.8 Integrity axis

$$Integrity3 = \{LOGOS, NOMOS, PATHOS\}$$

The compact Hamming arrangement is:

```
[LOGOS NOMOS FS PATHOS GS RS US]
```

The even-parity equations are:

$$LOGOS = FS \oplus GS \oplus US$$

$$NOMOS = FS \oplus RS \oplus US$$

$$PATHOS = GS \oplus RS \oplus US$$

Scope carries the data. Integrity checks the carried relation.

5.9 Temporal-integrity routing

The canonical diagonal routing is:

```
LL → LOGOS

MM → NOMOS

NN → PATHOS
```

This is not an identity between time and integrity.

It is a selected routing through the complete temporal-integrity matrix:

$$Delta3 \times Integrity3$$

The complete architecture keeps scope, time, integrity, and observer-relative projection distinct.

5.10 Carrier axis

$$Carrier = \{LowControl, LowAffine, HighControl, HighProjective\}$$

The carrier axis determines how a relation is encoded or exposed.

It does not determine whether the relation is valid.

5.11 Projection axis

$$Projection = \{Character, Bitboard, Canvas, Port, Azimuth, Debug, None\}$$

Projection is an inspectable view of a relation.

Projection must not become source authority.

5.12 Capability axis

$$Capability = \{Pure, Memory, Storage, Network, Clock, Foreign, Projection\}$$

A declaration that requires an effect must state the capability through which that effect can occur.

There is no ambient unrestricted host authority.

---

6. Canonical homoiconic syntax

6.1 Canonical S-expressions

The canonical syntax is the S-expression.

```
(module geometry.o
  (scope GS)
  (origin remote)
  (define project-centroid
    (expr
      (lambda (tetrahedron)
        (cons-centroid tetrahedron)))))
```

A canonical S-expression is both:

```
program structure

runtime data
```

This establishes homoiconicity.

6.2 Core syntax

A minimal abstract grammar is:

```
Expr        ::= Nil
              | Atom
              | Pair
              | Quote
              | Quasiquote
              | Unquote
              | Module
              | Declaration
              | MetaDeclaration

Pair        ::= "(" Expr "." Expr ")"
              | "(" {Expr} ")"

Declaration
            ::= "(" "define" Symbol Expr ")"
```

```
              | "(" "bind" BindingClass Symbol Expr ")"

Module      ::= "(" "module" ModuleId {ModuleField} ")"

ModuleField
            ::= "(" "origin" Origin ")"
              | "(" "scope" Scope ")"
              | "(" "requires" {Capability} ")"
              | Declaration
```

6.3 Proper and improper lists

A proper list:

```
(A B C)
```

has canonical structure:

```
(A . (B . (C . NIL)))
```

An improper list:

```
(A B . C)
```

has canonical structure:

```
(A . (B . C))
```

Renderers must preserve this distinction.

6.4 Readable M-expressions

The readable M-expression surface is intended for human-facing operator notation.

```
PROJECTIVE[
  CONS_FS(CAR_FS, CDR_FS) :
  CONS_GS(CAR_GS, CDR_GS) :
  CONS_RS(CAR_RS, CDR_RS) :
```

```
  CONS_US(CAR_US, CDR_US)
]
```

It lowers to:

```
(projective
  (FS . (cons CAR_FS CDR_FS))
  (GS . (cons CAR_GS CDR_GS))
  (RS . (cons CAR_RS CDR_RS))
  (US . (cons CAR_US CDR_US)))
```

Canonical law:

```
MEXPR presents.

SEXPR preserves.

Evaluation operates on canonical structure.
```

M-expression syntax must never create semantic forms that have no canonical S-expression lowering.

---

7. Meta-CONS

7.1 Structural CONS

At the base level:

$$CONS : A \times B \to Pair(A, B)$$

with:

$$CAR(CONS(a, b)) = a$$

$$CDR(CONS(a, b)) = b$$

7.2 Resolver CONS

At the resolver level:

$$CONS_r : CAR_r \times CDR_r \to Result_r$$

where r identifies a resolver profile.

Different resolver profiles may implement:

```
ordinary pair construction

affine quasigroup combination

scope-specific composition

bitboard combination

centroid reduction

carrier normalization
```

7.3 Quasigroup CONS

A quasigroup Q is a set equipped with a binary operation $*$ such that for every $a, b \in Q$, the equations

$$a * x = b$$

and

$$y * a = b$$

have unique solutions.

For Meta-CONS:

$$CAR * CDR = CONS$$

with recovery operations:

$$CAR \backslash CONS = CDR$$

$$CONS / CDR = CAR$$

The resolver law therefore permits any missing coordinate to be recovered from the other two.

The useful computational pattern is exactly: given two bounded coordinates, resolve or recover the third.

7.4 Resolver law requirements

A lawful quasigroup profile must satisfy:

Left recovery

$$a \backslash (a * b) = b$$

Right recovery

$$(a * b) / b = a$$

Left reconstruction

$$a * (a \backslash c) = c$$

Right reconstruction

$$(c / b) * b = c$$

Determinism

$$a = b \land c = d \Rightarrow a * c = b * d$$

Totality within profile domain

For every valid pair of profile coordinates, resolution returns exactly one valid result.

7.5 Why plain XOR is insufficient as the full structural identity

XOR is useful for:

```
bounded masks

delta computation

syndromes

complementary witnesses

involutions
```

But XOR is commutative:

$$a \oplus b = b \oplus a$$

A structural CONS relation is ordered.

Therefore plain XOR cannot distinguish:

```
(CAR . CDR)

(CDR . CAR)
```

without additional tags or asymmetric transforms.

A possible affine quasigroup family is:

$$a * b = P(a) \oplus Q(b) \oplus k$$

where P and Q are distinct invertible transformations and k is a profile constant.

Recovery is then:

$$a \backslash c = Q^{-1}(P(a) \oplus c \oplus k)$$
$$c / b = P^{-1}(c \oplus Q(b) \oplus k)$$

This preserves bounded reversible computation while retaining order.

---

8. Tetrahedral scope object

8.1 Scope vertices

Let the regular tetrahedral scope object have vertices:

$$A = FS$$
$$B = GS$$
$$C = RS$$
$$D = US$$

Each vertex carries one scoped relation:

$$X_{FS} = CONS_{FS}(CAR_{FS}, CDR_{FS})$$
$$X_{GS} = CONS_{GS}(CAR_{GS}, CDR_{GS})$$
$$X_{RS} = CONS_{RS}(CAR_{RS}, CDR_{RS})$$

$$X_{US} = CONS_{US}(CAR_{US}, CDR_{US})$$

8.2 Projective scope coordinate

The unresolved projective scope state is:

$$P = [X_{FS} : X_{GS} : X_{RS} : X_{US}]$$

or:

```
P =
[CONS_FS : CONS_GS : CONS_RS : CONS_US]
```

This notation preserves the four scope contributions simultaneously.

8.3 Edges

The six pairwise scope relations are:

```
FS-GS

FS-RS

FS-US

GS-RS

GS-US

RS-US
```

8.4 Faces

The four three-way scope surfaces are:

```
FS-GS-RS

FS-GS-US
```

```
FS-RS-US

GS-RS-US
```

8.5 Centroid

The centroid is not a fifth scope.

It is the balanced relation induced by the active four-scope configuration:

$$C_{meta} = Centroid(X_{FS}, X_{GS}, X_{RS}, X_{US})$$

Canonical law:

```
CONS is the centroid reducer.

CONS is not a fifth vertex.
```

---

9. Coproduct blackboard

9.1 Independent knowledge sources

Let:

$$B_i \subseteq P_{256}$$

be a sparse board contributed by source i, where:

$$P_{256} = \{0, \ldots, 255\}$$

A source may be:

```
a local module

a remote module

a proof package

a rule module

a facts module
```

```
an embedded device

a generated adapter

a recovered historical state
```

9.2 Origin-tagged disjoint union

The complete contribution domain is not an ordinary untagged union.

It is:

$$\mathcal{B} = \bigsqcup_{i \in I} B_i$$

Each contribution is represented as:

$$(i, x)$$

where:

```
i
is source identity

x
is the source-local coordinate
```

This prevents equal visible coordinates from erasing origin.

9.3 Coproduct injections

For each source i, there is an injection:

$$\iota_i : B_i \to \mathcal{B}$$

The injection preserves:

```
origin

scope
```

```
binding

CAR

CDR

stage

temporal coordinate

integrity data

carrier metadata

version
```

9.4 Shared plane projection

Meta-CONS defines a mediating map:

$$\mu : \bigsqcup_{i \in I} B_i \to P_{256}$$

The map produces the shared visible plane.

The mapping may be:

```
injective for non-overlapping contributions

fiber-producing for compatible overlaps

rejecting for incompatible overlaps

recovery-triggering for incomplete relations
```

9.5 Fiber preservation

For each visible coordinate $p \in P_{256}$, the runtime retains its fiber:

$$\mu^{-1}(p)$$

The visible coordinate therefore does not erase:

```
which sources contributed

which scopes were active

which CAR/CDR relations existed

which resolver was used

which declaration stage produced the view
```

Canonical law:

```
The byte is the view.

The tagged relation is the identity-bearing runtime object.
```

---

10. Sparse 256-bit boards

10.1 Rank-8 input domain

A rank-8 Boolean input domain contains:

$$2^8 = 256$$

possible input assignments.

One selected predicate over that input domain is a 256-bit truth table.

The complete population contains:

$$2^{256}$$

possible predicates, but the runtime stores selected predicates rather than instantiating that population.

10.2 Board representation

A portable implementation may use:

```
newtype Board256 =
  Board256 (Word64, Word64, Word64, Word64)
```

or:

```
typedef struct {
    uint64_t lane[4];
} MetaConsBoard256;
```

10.3 Board laws

A lawful board implementation must provide:

```
exactly 256 addressable bits

deterministic indexing

bounded union

bounded intersection

bounded symmetric difference

origin-preserving composition

canonical serialization
```

---

11. Binding and environment model

11.1 Scoped environments

For symbol s and scope σ:

$$E_\sigma(s)$$

may contain:

$$[CAR, CDR, APVAL, APVAL1, SUBR, FSUBR, EXPR, FEXPR]$$

The complete environment is:

$$E(s) = [E_{FS}(s) : E_{GS}(s) : E_{RS}(s) : E_{US}(s)]$$

This gives a four-scope by eight-class environment surface.

11.2 Value lookup

When a symbol appears in value position, the resolver searches admissible:

```
APVAL

APVAL1
```

bindings according to:

```
scope

origin

stage

capability

carrier policy
```

11.3 Operator lookup

When a symbol appears in operator position, the resolver selects among:

```
SUBR

FSUBR

EXPR

FEXPR
```

The selected class determines:

```
system versus user origin
```

```
ordinary versus special evaluation

whether operands are evaluated

which capabilities are required

which lowering rule applies
```

11.4 Dispatch function

The abstract dispatch function is:

$$Dispatch : Symbol \times Scope \times Position \times Stage \times OriginPolicy \to Binding$$

It may fail with an explicit ambiguity or missing-binding error.

It must not silently select one of several incomparable candidates.

---

12. Multi-stage metaprogramming

12.1 Stage progression

The compilation pipeline is:

```
SurfaceExpr

↓

ParsedExpr

↓

ExpandedExpr

↓

TypedExpr

↓

NormalizedExpr
```

```
↓

ResolvedExpr

↓

CoreProgram

↓

TargetProgram
```

12.2 Quotation

Quotation prevents evaluation:

```
(quote (a b c))
```

The result remains canonical syntax data.

12.3 Quasiquotation

Quasiquotation constructs syntax while permitting controlled insertion:

```
`(cons ,car-form ,cdr-form)
```

12.4 Macros

Macros transform canonical syntax before runtime evaluation.

A macro:

```
must receive syntax

must return syntax

must preserve source-location information where available
```

```
must not execute undeclared host effects
```

12.5 FEXPRs

FEXPRs receive unevaluated operand forms during evaluation.

They differ from macros:

```
macro
transforms code before ordinary evaluation

FEXPR
participates in runtime evaluation while controlling operand evaluation
```

A bounded FEXPR design requires explicit evaluation policy and capability checks to prevent invisible host-language control.

12.6 Staging safety

A form at stage s_2 may depend only on declarations available at an admissible earlier or equal stage.

A type-level staging relation should reject illegal backward references.

---

13. Metadata, meta-knowledge, and metalogic

13.1 Metadata

Metadata describes a declaration:

```
(meta
  (origin . geometry.o)
  (scope . GS)
  (binding . EXPR)
  (stage . Typed)
  (capabilities . (Pure Projection)))
```

Metadata is ordinary canonical data, but the runtime recognizes a typed schema for it.

13.2 Meta-knowledge

Meta-knowledge represents knowledge about the state of another relation.

Examples include:

```
relation is defined

relation is unresolved

relation was recovered

relation has passed integrity checking

relation was lowered

relation has a projection

relation requires unavailable capability
```

These states must not be collapsed into a single Boolean “valid” field.

13.3 Metalogic

Metalogic is the language-level representation of logic operators, inference rules, evaluation policies, and proof obligations.

A metalogical object can describe:

```
what an operator means

which operands it accepts

which result type it produces

which integrity law applies

which proof obligation remains
```

It does not itself constitute a proof unless checked by an appropriate proof authority.

---

14. Meta-circular evaluation

14.1 Reference evaluator

A reference evaluator may be expressed in Emergent Axial Lisp itself.

```
(define eval
  (fexpr (form env)
    (cond
      ((atom? form)
       (resolve-atom form env))
      ((special-form? (car form) env)
       (apply-special (car form) (cdr form) env))
      (else
       (apply
         (eval (car form) env)
         (map-eval (cdr form) env))))))
```

This demonstrates meta-circularity.

14.2 Trust boundary

The self-described evaluator is not automatically the trusted kernel.

The trusted base should remain smaller:

```
canonical parser

core data constructors

type checker

stage checker

primitive resolver

capability gate

target verifier
```

The meta-circular evaluator is then tested against that reference kernel.

---

15. Meta-memory

15.1 Memory as coordinated relation

A memory entry is not merely:

```
address → bytes
```

It is:

$$MemoryEntry = (value, origin, scope, binding, time, integrity, carrier, stage, ancestry)$$

15.2 Meta-memory operations

The runtime should support:

```
inspect entry metadata

trace origin

recover missing coordinate

compare temporal states

project selected axes

serialize canonical relation

replay transformation history
```

15.3 No hidden ambient heap identity

A host pointer may locate an implementation object, but it is not the canonical identity of the language object.

Canonical identity must be reconstructible from declared runtime structure and serialization.

---

16. Meta-compiler

16.1 Compiler passes as objects

Compiler passes are represented as typed metaobjects:

```
ParserPass

MacroExpansionPass

TypeElaborationPass

NormalizationPass

ResolutionPass

LoweringPass

TargetEmissionPass
```

Each pass declares:

```
input stage

output stage

required capabilities

preserved invariants

possible errors

proof obligations
```

16.2 Compiler architecture

```
Emergent Axial Lisp

↓

Canonical SEXPR
```

```
↓

Multi-Axial Elaboration

↓

Meta-CONS Core IR

↓

Normalization and Recovery

↓

Target Lowering

↓

Omnicron ISA / C / WASM / embedded carrier
```

16.3 Meta-CONS Core IR

The Core IR should be smaller than the surface language.

A minimal form may include:

```
CoreNil

CoreAtom

CorePair

CoreBind

CoreLookup

CoreLambda

CoreApply

CoreSpecial

CoreQuote
```

```
CoreScope

CoreRecover

CoreProject

CoreCapability
```

M-expression syntax, documentation syntax, Canvas syntax, and other adapters must lower into this shared core rather than defining independent execution laws.

---

17. Relation to AAL

17.1 AAL as predecessor

The Assembly–Algebra Language lineage demonstrates a useful architecture consisting of:

```
parser

AST

well-formedness checking

graded typing

small-step semantics

polynomial arithmetic

geometry mapping

compiler

interpreter

target generation

proof obligations
```

This is the appropriate role from which Emergent Axial Lisp should borrow.

17.2 What is retained

Emergent Axial Lisp retains these AAL principles:

```
typed operational semantics

explicit abstraction levels

deterministic lowering

algebraic interpretation

proof-obligation generation

separation of source and target semantics
```

17.3 What changes

AAL is assembly-facing and algebra-facing.

Emergent Axial Lisp is:

```
homoiconic

declarative

module-oriented

origin-preserving

reflective

multi-stage

multi-axial

capability-bounded
```

17.4 Evidence boundary

The AAL corpus contains substantial implementation evidence in Scheme and TypeScript and several formal-model artifacts. An internal inventory also warns that archived Coq files contain admits and placeholders and therefore require audit before being promoted as active proof authority.

Accordingly, Emergent Axial Lisp must distinguish:

```
documented claim

implemented behavior

tested behavior

formally proved theorem
```

No inherited AAL claim becomes a theorem of Emergent Axial Lisp merely by citation.

---

18. Pure typed Haskell reference model

18.1 Design rule

The Haskell model should not encode every concept as a type class.

Use:

```
algebraic data types
for closed facts

GADTs
for stage-indexed and type-indexed syntax

type families
for legal transitions and computed relationships

newtypes
for bounded values and invariants

type classes
for open behavior
```

```
existentials
only at controlled extension boundaries
```

18.2 Promoted axes

```
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE GADTs #-}
{-# LANGUAGE KindSignatures #-}
{-# LANGUAGE TypeFamilies #-}
{-# LANGUAGE MultiParamTypeClasses #-}
{-# LANGUAGE FunctionalDependencies #-}
{-# LANGUAGE FlexibleInstances #-}
{-# LANGUAGE GeneralizedNewtypeDeriving #-}
{-# LANGUAGE DerivingStrategies #-}
{-# LANGUAGE StandaloneDeriving #-}
{-# LANGUAGE RankNTypes #-}
{-# LANGUAGE ExistentialQuantification #-}
```

```
data ScopeAxis
  = FS
  | GS
  | RS
  | US

data StructuralAxis
  = Car
  | Cdr
  | Cons

data BindingAxis
  = BCar
  | BCdr
  | APVal
  | APVal1
  | Subr
  | FSubr
  | Expr
  | FExpr

data EvalAxis
  = Eager
  | Lazy
  | Special
```

```
  | Macro
  | Reflective
  | Quoted

data StageAxis
  = Surface
  | Parsed
  | Expanded
  | Typed
  | Normalized
  | Resolved
  | Lowered

data TimeAxis
  = LL
  | MM
  | NN

data IntegrityAxis
  = Logos
  | Nomos
  | Pathos

data CarrierAxis
  = LowControl
  | LowAffine
  | HighControl
  | HighProjective

data ProjectionAxis
  = NoProjection
  | CharacterProjection
  | BitboardProjection
  | CanvasProjection
  | PortProjection
  | AzimuthProjection
```

18.3 Singleton witnesses

Runtime-loaded data needs singleton witnesses for promoted coordinates.

```
data SScope (s :: ScopeAxis) where
  SFS :: SScope 'FS
  SGS :: SScope 'GS
```

```
  SRS :: SScope 'RS
  SUS :: SScope 'US
```

Comparable witnesses should exist for every closed axis that crosses a runtime serialization boundary.

18.4 Bounded primitive types

```
newtype ByteCoord =
  ByteCoord Word8

newtype Car32 =
  Car32 Word32

newtype Cdr32 =
  Cdr32 Word32

newtype OriginId =
  OriginId ByteString

newtype Symbol =
  Symbol ByteString

newtype Board256 =
  Board256 (Word64, Word64, Word64, Word64)
```

Constructors should be hidden when an invariant requires validation.

18.5 Stage-indexed syntax

```
data Term
  (stage :: StageAxis)
  (scope :: ScopeAxis)
  (binding :: BindingAxis)
  where

  TNil
    :: Term stage scope binding

  TSymbol
    :: Symbol
    -> Term stage scope binding
```

```
  TPair
    :: Term stage scope binding
    -> Term stage scope binding
    -> Term stage scope binding

  TQuote
    :: Term 'Parsed scope binding
    -> Term 'Expanded scope binding

  TScoped
    :: SScope inner
    -> Term stage inner binding
    -> Term stage outer binding
```

The actual production type will likely separate term syntax from declaration metadata to reduce excessive phantom parameters, but the formal model should preserve the coordinate relation explicitly.

18.6 Existential declarations

Modules can contain declarations whose type-level coordinates differ.

```
data SomeDeclaration =
  forall scope binding.
  SomeDeclaration
    (SScope scope)
    (SBinding binding)
    (Declaration scope binding)
```

The existential package must carry all dictionaries and witnesses needed for safe dispatch.

18.7 Type families

Evaluation policy

```
type family EvalPolicy
  (binding :: BindingAxis)
  :: EvalAxis where

  EvalPolicy 'Subr  = 'Eager
  EvalPolicy 'Expr  = 'Eager
  EvalPolicy 'FSubr = 'Special
  EvalPolicy 'FExpr = 'Special
```

```
  EvalPolicy 'APVal = 'Quoted
  EvalPolicy 'APVal1 = 'Quoted
```

Mirror carrier

```
type family Mirror
  (carrier :: CarrierAxis)
  :: CarrierAxis where

  Mirror 'LowControl     = 'HighControl
  Mirror 'LowAffine      = 'HighProjective
  Mirror 'HighControl    = 'LowControl
  Mirror 'HighProjective = 'LowAffine
```

Next stage

```
type family NextStage
  (stage :: StageAxis)
  :: StageAxis where

  NextStage 'Surface    = 'Parsed
  NextStage 'Parsed     = 'Expanded
  NextStage 'Expanded   = 'Typed
  NextStage 'Typed      = 'Normalized
  NextStage 'Normalized = 'Resolved
  NextStage 'Resolved   = 'Lowered
```

`Lowered` has no implicit next stage.

18.8 Open behavioral classes

```
class Resolvable a where
  type Resolved a
  resolve
    :: a
    -> Either ResolveError (Resolved a)
```

```
class Reflective a where
  metaobject
    :: a
    -> MetaObject
```

```
class Lowerable source target where
  lower
    :: source
    -> Either LowerError target
```

```
class Projectable source projection where
  project
    :: source
    -> Either ProjectionError projection
```

18.9 Quasigroup class

```
class Quasigroup q where
  qmul
    :: q
    -> q
    -> q

  leftDivide
    :: q
    -> q
    -> q

  rightDivide
    :: q
    -> q
    -> q
```

Required laws:

```
leftDivide a (qmul a b) == b

rightDivide (qmul a b) b == a

qmul a (leftDivide a c) == c

qmul (rightDivide c b) b == c
```

These laws should be tested with exhaustive finite-domain tests where feasible and proven in Coq for normative resolver profiles.

---

19. Pure semantic core

19.1 Semantic functions

The central language pipeline should remain pure:

```
parse
  :: SurfaceSource
  -> Either ParseError ParsedModule
```

```
expand
  :: ExpansionEnv
  -> ParsedModule
  -> Either ExpansionError ExpandedModule
```

```
elaborate
  :: TypeEnv
  -> ExpandedModule
  -> Either TypeError TypedModule
```

```
normalize
  :: TypedModule
  -> Either NormalizeError NormalizedModule
```

```
resolveModule
  :: ResolverEnv
  -> NormalizedModule
  -> Either ResolveError ResolvedModule
```

```
lowerModule
  :: Target target
  -> ResolvedModule
  -> Either LowerError (Program target)
```

19.2 Why purity matters

Keeping these phases pure provides:

```
deterministic replay

property testing

referential transparency

small trusted interfaces

portable embedding

formal correspondence with Coq

independence from host IO
```

---

20. Effectful Meta-CONS host runtime

20.1 Transformer stack

Effects belong in a separate host layer.

```haskell
newtype MetaConsT m a =
  MetaConsT
    { unMetaConsT ::
        ReaderT RuntimeCapabilities
          (StateT RuntimeState
            (ExceptT RuntimeError
              (WriterT [RuntimeEvent] m))) a
    }
  deriving newtype
    ( Functor
    , Applicative
    , Monad
    , MonadReader RuntimeCapabilities
    , MonadState RuntimeState
    , MonadError RuntimeError
    , MonadWriter [RuntimeEvent]
    )
```

20.2 Runtime capabilities

```
data RuntimeCapabilities = RuntimeCapabilities
  { capReadModule
      :: ModuleRef
      -> IO ByteString

  , capWriteProjection
      :: Projection
      -> IO ()

  , capResolveRemote
      :: RemoteRequest
      -> IO RemoteResult

  , capClock
      :: IO LogicalTime
  }
```

A pure profile may supply no effectful capabilities.

20.3 Capability discipline

The runtime must enforce:

```
no undeclared filesystem access

no undeclared network access

no hidden clock access

no ambient foreign-function access

no projection write without projection capability
```

A missing capability produces an explicit runtime error.

---

21. Metaobject protocol

21.1 Metaobject structure

```
data MetaObject = MetaObject
  { moClass
      :: ObjectClass

  , moScope
      :: SomeScope

  , moBinding
      :: SomeBinding

  , moOrigin
      :: OriginId

  , moStage
      :: SomeStage

  , moEvaluation
      :: EvalPolicyValue

  , moTime
      :: Maybe TimeValue

  , moIntegrity
      :: Maybe IntegrityValue

  , moCarrier
      :: Maybe CarrierValue

  , moRecovery
      :: Maybe RecoveryProfileId

  , moCapabilities
      :: Set Capability
  }
```

21.2 Reflective operations

The protocol exposes:

```
class-of

scope-of

binding-of

origin-of

stage-of

evaluation-policy-of

time-of

integrity-of

carrier-of

recovery-law-of

lowering-of

projection-of

capabilities-of
```

21.3 Reflective mutation

Inspection is permitted more broadly than mutation.

Changing a metaobject requires:

```
a declared reflective capability

a valid stage transition

preservation of canonical identity

revalidation of affected invariants
```

Reflection must not become unrestricted self-modifying host access.

---

22. Formal judgments

22.1 Typing judgment

A core typing judgment is:

$$\Gamma; \Sigma; \Omega; \Delta \vdash e : \tau @ s$$

where:

```
Γ
value and operator environment

Σ
scope environment

Ω
origin environment

Δ
stage, temporal, and capability context

τ
result type

s
stage
```

22.2 Resolution judgment

$$\Gamma; \Sigma; \Omega \vdash symbol \Downarrow binding$$

This judgment succeeds only when one admissible binding is selected.

22.3 Stage judgment

$$e @ s_1 \rightsquigarrow e' @ s_2$$

with:

$$s_2 = NextStage(s_1)$$

unless an explicitly typed pass defines another lawful transition.

22.4 Recovery judgment

$$r \vdash CAR, CDR \Downarrow CONS$$

$$r \vdash CAR, CONS \Downarrow CDR$$

$$r \vdash CONS, CDR \Downarrow CAR$$

where r is a resolver profile satisfying the quasigroup laws.

22.5 Coproduct composition judgment

$$\{\iota_i(B_i)\}_{i \in I} \vdash_\mu P_{256}$$

The result must preserve a retrievable source fiber for every projected coordinate.

---

23. Safety and correctness properties

The formal project should target the following theorems.

23.1 Parser determinism

If:

$$parse(x) = a$$

and:

$$parse(x) = b$$

then:

$$a = b$$

23.2 M-expression lowering determinism

If:

$$lower_M(m) = s_1$$

and:

$$lower_M(m) = s_2$$

then:

$$s_1 = s_2$$

23.3 Type preservation

If:

$$\Gamma \vdash e : \tau$$

and:

$$e \to e'$$

then:

$$\Gamma \vdash e' : \tau$$

subject to the stage-indexed semantics.

23.4 Progress

A well-typed closed resolved term is:

```
a value

or can make one semantic step

or produces a declared capability requirement

or produces an explicit bounded error
```

It does not become silently stuck.

23.5 Stage preservation

A pass cannot emit a term claiming a stage that does not correspond to its output invariants.

23.6 Origin preservation

Coproduct composition cannot erase source identity from the internal fiber even when several contributions share one visible coordinate.

23.7 Recovery correctness

For every lawful resolver profile:

$$leftDivide(a, qmul(a, b)) = b$$

and:

$$rightDivide(qmul(a, b), b) = a$$

23.8 Mirror involution

For every byte coordinate:

$$(x \oplus 0x80) \oplus 0x80 = x$$

23.9 Scope-integrity correctness

The Hamming syndrome must identify:

```
no error

or one inconsistent position among
LOGOS, NOMOS, FS, PATHOS, GS, RS, US
```

without treating temporal Delta3 positions as syndrome bits.

23.10 Projection non-authority

No projection operation may change the accepted canonical relation unless it invokes a separately authorized declaration or update operation.

---

24. Validation, coordinates, and historical “receipts”

The architecture should prefer precise role names.

Where older documents use “receipt” for several different things, the runtime should distinguish:

```
coordinate
where a relation is placed

witness
evidence that a computation or relation was observed

attestation
a signed or declared statement about a relation

acceptance
the decision that a relation is admissible

record
the persisted representation of accepted state

projection
an inspectable view
```

“Receipt” may remain as a compatibility or historical term, but it must not collapse all of these roles.

---

25. Authority model

The architecture observes the following separation.

```
Syntax proposes structure.

Typing establishes well-formedness.

Resolution selects a binding.

Quasigroup recovery reconstructs a coordinate.

Integrity detects bounded inconsistency.
```

```
Validation decides admissibility.

Acceptance changes canonical state.

Recording preserves accepted state.

Projection exposes a view.

Reflection inspects the mechanism.

Capability controls effects.
```

No one line automatically grants the authority of another.

---

26. Embedding interface

26.1 Portable host boundary

The host supplies explicit functions.

```c
typedef struct MetaConsHost {
    void *context;

    MetaConsStatus (*read_module)(
        void *context,
        MetaConsModuleId id,
        MetaConsSlice *out
    );

    MetaConsStatus (*resolve_remote)(
        void *context,
        const MetaConsRemoteRequest *request,
        MetaConsRemoteResult *out
    );

    MetaConsStatus (*write_projection)(
        void *context,
        const MetaConsProjection *projection
    );
} MetaConsHost;
```

26.2 Core runtime restrictions

The portable core should require only:

```
fixed-width integer operations

bounded allocation

canonical byte serialization

ordered pair construction

deterministic parser and evaluator

registered host capabilities
```

26.3 Supported environments

The architecture can be embedded in:

```
native command-line programs

servers

browser WebAssembly

microcontrollers

firmware tools

offline notebooks

peer-to-peer modules

graphical editors

proof-assistant extraction targets
```

The environment changes the available capabilities, not the language's canonical semantic rules.

---

27. Module system

27.1 Module declaration

```
(module geometry.o
  (origin "did:example:geometry")
  (scope GS)
  (provides
    project-centroid
    tetrahedron-incidence)
  (requires
    Pure
    Projection)
  ...)
```

27.2 Imports as coproduct injections

Import does not copy an untagged namespace into the current one.

Import establishes a tagged injection:

$$\iota_{geometry} : Geometry \to CurrentEnvironment$$

27.3 Conflict handling

When two imports define the same symbol, the runtime may:

```
require qualified lookup

apply an explicit precedence declaration

construct a composite binding

reject the ambiguity
```

It must not silently choose by load order unless the module explicitly declares load-order semantics.

---

28. Serialization

28.1 Canonical serialization

Canonical serialization must preserve:

```
pair order

proper versus improper lists

scope

binding

origin

stage

temporal and integrity annotations

resolver profile

board coordinates

extension fields
```

28.2 Semantic and projection digests

Two digests should be distinguished.

```
semantic digest
covers canonical runtime identity

projection digest
covers one rendered or carrier-specific representation
```

Changing whitespace or Canvas placement may change the projection digest without changing semantic identity.

---

29. Conformance architecture

A conforming implementation should provide:

```
parser vectors

M-expression lowering vectors

scope vectors

binding dispatch vectors

Hamming [7,4,3] vectors

Delta3 vectors

quasigroup recovery vectors

coproduct provenance vectors

board composition vectors

carrier mirror vectors

projection non-authority tests

serialization round trips

negative compile-time tests
```

The existing Omnicron conformance work already separates scope, Delta3, activation, projection, and COBS profiles, which supplies a useful foundation for this test organization.

---

30. Repository architecture

A clean standalone repository may use:

```
emergent-axial-lisp/
├── README.md
├── CHARTER.md
├── LANGUAGE-SPEC.md
├── META-CONS-RUNTIME.md
```

```
├── METAOBJECT-PROTOCOL.md
├── MULTI-AXIAL-TYPE-MODEL.md
├── CONFORMANCE.md
├── SECURITY.md
├── PORTABILITY.md
│
├── src/
│   ├── Emergent/
│   │   ├── Syntax.hs
│   │   ├── Parser.hs
│   │   ├── MExpr.hs
│   │   ├── Macro.hs
│   │   ├── Type.hs
│   │   └── Module.hs
│   │
│   ├── MetaCons/
│   │   ├── Kernel.hs
│   │   ├── Axis.hs
│   │   ├── Environment.hs
│   │   ├── Resolver.hs
│   │   ├── Quasigroup.hs
│   │   ├── Coproduct.hs
│   │   ├── Bitboard.hs
│   │   ├── Reflect.hs
│   │   ├── Evaluate.hs
│   │   └── Lower.hs
│   │
│   └── Adapter/
│       ├── Markdown.hs
│       ├── Canvas.hs
│       ├── Port.hs
│       └── OmnicronISA.hs
│
├── proof/
│   ├── Syntax.v
│   ├── Stages.v
│   ├── Scope.v
│   ├── Integrity.v
│   ├── Quasigroup.v
│   ├── Coproduct.v
│   └── Preservation.v
│
├── vectors/
│   ├── syntax.yaml
│   ├── scope4.yaml
│   ├── compact743.yaml
```

```
│   ├── delta3.yaml
│   ├── recovery.yaml
│   ├── coproduct.yaml
│   └── carrier-mirror.yaml
│
├── examples/
│   ├── hello.eal
│   ├── scoped-module.eal
│   ├── fexpr.eal
│   ├── coproduct-board.eal
│   └── reflective-evaluator.eal
│
└── test/
    ├── Properties.hs
    ├── Golden.hs
    ├── Negative/
    └── Conformance/
```

---

31. Development phases

Phase 0 — Canonical language charter

Freeze:

```
language name

runtime name

axis names

authority boundaries

canonical SEXPR role

MEXPR lowering rule
```

Phase 1 — Pure kernel

Implement:

```
Null

Atom

Pair

CAR

CDR

canonical serializer

bounded byte and word types
```

Phase 2 — Multi-axial types

Implement promoted types and singleton witnesses for:

```
scope

binding

evaluation

stage

time

integrity

carrier

projection
```

Phase 3 — Parser and canonical AST

Implement:

```
S-expression parser
```

```
proper and improper lists

quotation

source spans

canonical printer

round-trip tests
```

Phase 4 — Environment and dispatch

Implement:

```
APVAL/APVAL1 lookup

SUBR/FSUBR/EXPR/FEXPR dispatch

scope-local lookup

origin-qualified lookup

ambiguity errors
```

Phase 5 — Quasigroup recovery

Implement:

```
resolver profile interface

one finite reversible profile

left and right division

exhaustive finite-domain tests

Coq proof of profile laws
```

Phase 6 — Coproduct blackboard

Implement:

```
origin-tagged source injection

Board256

fiber-preserving projection

conflict reporting

canonical board serialization
```

Phase 7 — Multi-stage evaluation

Implement:

```
macro expansion

FEXPR evaluation

typed normalization

stage checking

core IR
```

Phase 8 — Metaobject protocol

Implement inspection before mutation:

```
class-of

scope-of

binding-of

origin-of
```

```
stage-of

evaluation-policy-of

recovery-law-of
```

Phase 9 — Effect runtime

Implement:

```
MetaConsT

capability registry

module loader

remote resolver adapter

projection writer
```

Phase 10 — Lowering targets

Implement:

```
Omnicron ISA target

portable C target

WASM target

debug interpreter target
```

Phase 11 — Formal verification

Prove:

```
parser and lowering determinism
```

```
stage preservation

type preservation

progress

quasigroup recovery

origin preservation

projection non-authority properties
```

---

32. Non-goals

Emergent Axial Lisp is not defined as:

```
a replacement for Coq

a physical quantum computer

a claim that byte tables are inherently projective planes

a claim that every metaphor is a theorem

an unrestricted self-modifying Lisp

a globally shared mutable symbol table

a network protocol with implicit authority

a projection or UI framework pretending to validate state
```

---

33. Normative terminology

```
Emergent Axial Lisp
the language

Multi-Axial Runtime Model
the typed coordinate architecture
```

```
Meta-CONS Runtime
the execution, resolution, recovery, and embedding engine

Meta-CONS Metaobject Protocol
the reflective interface to runtime structure

OMI
the foundational relational and notation doctrine

Omicron
the bounded gauge and character surface

Omnicron
the multi-plane resolver and orchestration architecture

AAL
the assembly-algebra predecessor and formal-method lineage
```

---

34. Canonical architecture

```
Emergent Axial Lisp
├── canonical SEXPR representation
├── readable MEXPR surface
├── quotation and quasiquotation
├── hygienic macros
├── FSUBR/FEXPR special evaluation
├── declarative modules
└── reflective metaobject declarations
```

```
Multi-Axial Runtime Model
├── FS / GS / RS / US scope
├── CAR / CDR / CONS structure
├── APVAL / APVAL1 value binding
├── SUBR / FSUBR system dispatch
├── EXPR / FEXPR user dispatch
├── local / module / remote origin
├── eager / special / macro evaluation
├── surface-to-lowered stages
├── LL / MM / NN temporal position
├── LOGOS / NOMOS / PATHOS integrity
├── affine / projective carrier coordinates
```

```
├── character / board / canvas projection
└── explicit runtime capabilities
```

```
Meta-CONS Runtime
├── environment construction
├── ordered CONS resolution
├── origin-preserving coproduct
├── sparse 256-bit board composition
├── quasigroup recovery
├── staged evaluation
├── typed normalization
├── capability dispatch
├── reflection
├── canonical serialization
└── portable embedding
```

---

35. Final formal statement

Emergent Axial Lisp is a homoiconic multi-axial declarative language whose canonical S-expression structures are interpreted by the Meta-CONS Runtime through independent typed axes of scope, structure, binding, origin, evaluation, stage, time, integrity, carrier, projection, and capability. Independent local or remote modules enter through origin-preserving coproduct injections and may contribute sparse 256-bit boards to a shared OMI-Lisp plane without losing the complete CAR/CDR relations behind visible coordinates. Meta-CONS extends ordinary pair construction with typed dispatch, reflective metaobjects, multi-stage normalization, and quasigroup recovery, allowing any missing coordinate among CAR, CDR, and CONS to be reconstructed under a lawful resolver profile. A pure typed Haskell kernel defines the reference semantics, while controlled capability interfaces provide portable effects and target lowering without making any carrier, projection, adapter, or host environment authoritative over canonical language identity.

---

36. Final lock

```
The language is Emergent Axial Lisp.
```

```
The runtime is Meta-CONS.
```

```
The architecture is multi-axial,
not fixed-tuple.
```

```
The canonical representation is SEXPR.
```

```
MEXPR is a deterministic readable lowering surface.
```

```
Scope is FS/GS/RS/US.
```

```
Time is LL/MM/NN.
```

```
Integrity is LOGOS/NOMOS/PATHOS.
```

```
CAR/CDR preserve ordered relation.
```

```
CONS resolves and recovers.
```

```
Origins enter by tagged coproduct injection.
```

```
The shared plane is a projection,
not the loss of provenance.
```

```
Closed coordinates are data types.
```

```
Legal transitions are GADTs and type families.
```

```
Open extension behavior is expressed by type classes.
```

```
Effects are capability-mediated.
```

```
Pure semantics precede the transformer stack.
```

```
Reflection exposes structure
without automatically granting mutation.
```

```
Validation, recording, and projection
remain distinct authority surfaces.
```

```
The historical fixed 8-tuple is no longer
the normative runtime object.
```

```
The environment emerges from
typed objects and lawful relations.
```